

Admission Software - Code Explanation (Hinglish)

Yeh folder ek bohot detailed OOPs (Object-Oriented Programming) based admission system design karta hai. Yeh University/College me multiple programs, departments, students, normalizers (scoring system ko normalize karne ke liye), aur merit generation manage karta hai.

1. Curriculum.h

Yeh class program/degree ka curriculum define karti hai, jisme semesters, credits, aur subjects ki list hoti hai.

```
#pragma once
#include <string>
#include <vector>

using namespace std;

class Curriculum {
    int totalSemesters;
    int totalCredits;
    vector<string> subjects;

public:
    Curriculum(int totalSemesters, int totalCredits, const vector<string>&
subjects)
        : totalSemesters(totalSemesters), totalCredits(totalCredits),
subjects(subjects) {}
};
```

2. Qualification.h

Student ki pass ki hui ek degree ki details store karta hai jaise passing year, marks (CGPA/Percentage), aur type ("HIGH_SCHOOL", "BACHELORS", "GATE"). GATE jaise special exams ki detail bhi store karta hai.

```
#pragma once
#include <string>

using namespace std;

class Qualification {
    string degreeName;
    double cgpaOrPercentage;
    string institution;
    int yearOfPassing;
    string type;
    bool specialExamQualified;
    double specialExamScore;
};
```

```

public:
    // Constructor parameters init karta hai
    Qualification(string degreeName, double cgpaOrPercentage, string
institution, int yearOfPassing, string type, bool specialExamQualified =
false, double specialExamScore = 0.0)
        : degreeName(degreeName), cgpaOrPercentage(cgpaOrPercentage),
institution(institution), yearOfPassing(yearOfPassing), type(type),
specialExamQualified(specialExamQualified),
specialExamScore(specialExamScore) {}

    double getCgpaOrPercentage() const { return cgpaOrPercentage; }
    bool isSpecialExamQualified() const { return specialExamQualified; }
    double getSpecialExamScore() const { return specialExamScore; }
    string getDegreeName() const { return degreeName; }
    string getInstitution() const { return institution; }
    int getYearOfPassing() const { return yearOfPassing; }
    string getType() const { return type; }
};

```

3. Student.h

Student ki personal details aur uske qualifications ki ek array (vector) store karta hai. Ek student ke multiple qualifications ho sakte hain jaise 12th + B.Tech + GATE.

```

#pragma once
#include <string>
#include <vector>
#include "Qualification.h"

using namespace std;

class Student {
    string id;
    string name;
    vector<Qualification> qualifications; // Array of degrees

public:
    Student(string id, string name) : id(id), name(name) {}

    // Nayi degree/qualification add karne ke liye function
    void addQualification(const Qualification& q) {
        qualifications.push_back(q);
    }

    // Specific type ("GATE", "BACHELORS") ki qualification search karna
    const Qualification* getQualification(const string& type) const {
        for (const auto& q : qualifications) {
            if (q.getType() == type) {
                return &q;
            }
        }
    }
};

```

```

        return nullptr;
    }

    string getId() const { return id; }
    string getName() const { return name; }
};

```

4. ScoreNormalizer.h

Ek interface (base class) jiska function marks ko normalize karta hai (e.g. 10 point CGPA ko percentage me convert karna).

```

#pragma once
class ScoreNormalizer {
public:
    virtual double normalize(double score) const = 0;
    virtual ~ScoreNormalizer() = default;
};

```

5. PassThroughNormalizer.h & IndianCGPANormalizer.h

Yeh dono classes `ScoreNormalizer` ko implement karti hain. `PassThroughNormalizer` exact same marks wapis bhejta hai (no change). `IndianCGPANormalizer` agar score ≤ 10.0 hai, to usko 9.5 se multiply karke percentage me convert karta hai.

```

// PassThroughNormalizer.h
#pragma once
#include "ScoreNormalizer.h"

class PassThroughNormalizer : public ScoreNormalizer {
public:
    double normalize(double score) const override {
        return score; // Koi conversion nahi
    }
};

// IndianCGPANormalizer.h
#pragma once
#include "ScoreNormalizer.h"

class IndianCGPANormalizer : public ScoreNormalizer {
public:
    double normalize(double score) const override {
        if (score <= 10.0) {
            return score * 9.5; // CGPA ko percentage mein convert karne ka
            Indian standard rule
        }
        return score;
    }
};

```

```

    }
};

```

6. EligibilityChecker.h & BasicEligibilityChecker.h

Eligibility check karne ke classes hain. `BasicEligibilityChecker` ensure karta hai ki given qualification (e.g., "HIGH_SCHOOL") mein student ka score normalizer ke hisab se minimum percentage requirement ko cross kare.

```

// EligibilityChecker.h
#pragma once
#include <string>
#include "Student.h"

using namespace std;

class EligibilityChecker {
public:
    virtual bool isEligible(const Student& student) const = 0;
    virtual string getRejectionReason(const Student& student) const = 0;
    virtual ~EligibilityChecker() = default;
};

// BasicEligibilityChecker.h
#pragma once
#include <string>
#include <memory>
#include "EligibilityChecker.h"
#include "ScoreNormalizer.h"

using namespace std;

class BasicEligibilityChecker : public EligibilityChecker {
    string requiredType; // Kaunsi degree chahiye apply karne ke liye
    double minPercentage; // Minimum kitne marks chahiye
    shared_ptr<ScoreNormalizer> normalizer;

public:
    BasicEligibilityChecker(string requiredType, double minPercentage,
        shared_ptr<ScoreNormalizer> normalizer)
        : requiredType(requiredType), minPercentage(minPercentage),
        normalizer(normalizer) {}

    // Check karta hai agar student allowed hai ya nahi
    bool isEligible(const Student& student) const override {
        const Qualification* q = student.getQualification(requiredType);
        if (!q) return false;

        double score = normalizer->normalize(q->getCgpaOrPercentage());
        return score >= minPercentage;
    }
}

```

```
// Reject hone ka reason batata hai string format mein
string getRejectionReason(const Student& student) const override {
    const Qualification* q = student.getQualification(requiredType);
    if (!q) return "Missing required qualification";

    double score = normalizer->normalize(q->getCgpaOrPercentage());
    if (score < minPercentage) {
        return "Minimum percentage not met. Required: " +
to_string(minPercentage) + "%, Found: " + to_string(score) + "%";
    }
    return "Eligible";
}
};
```

7. MeritCalculator.h, StandardMeritCalculator.h, & GatePreferenceMeritCalculator.h

Yeh classes final selection (Merit list) ke time students ko score assign karti hain, taaki highest score waala pehle select ho.

- `StandardMeritCalculator` ek given qualification ka normalized score deta hai.
- `GatePreferenceMeritCalculator` B.Tech ke score ka 60% aur GATE exam ke score ka 40% weightage lekar total marks deta hai (agar usne GATE clear kiya hai, otherwise 60% hi milta hai).

```
// MeritCalculator.h
#pragma once
#include "Student.h"

class MeritCalculator {
public:
    virtual double calculateMerit(const Student& student) const = 0;
    virtual ~MeritCalculator() = default;
};

// StandardMeritCalculator.h
#pragma once
#include <string>
#include <memory>
#include "MeritCalculator.h"
#include "ScoreNormalizer.h"

using namespace std;

class StandardMeritCalculator : public MeritCalculator {
    string baseQualification;
    shared_ptr<ScoreNormalizer> normalizer;

public:
    StandardMeritCalculator(string baseQualification,
shared_ptr<ScoreNormalizer> normalizer)
```

```

        : baseQualification(baseQualification), normalizer(normalizer) {}

    double calculateMerit(const Student& student) const override {
        const Qualification* q =
student.getQualification(baseQualification);
        if (!q) return 0.0;

        return normalizer->normalize(q->getCgpaOrPercentage()); // Normal
marks ko as merit return karta hai
    }
};

// GatePreferenceMeritCalculator.h
#pragma once
#include <memory>
#include "MeritCalculator.h"
#include "ScoreNormalizer.h"

using namespace std;

class GatePreferenceMeritCalculator : public MeritCalculator {
    shared_ptr<ScoreNormalizer> normalizer;

public:
    GatePreferenceMeritCalculator(shared_ptr<ScoreNormalizer> normalizer) :
normalizer(normalizer) {}

    // GATE score calculation
    double calculateMerit(const Student& student) const override {
        const Qualification* bachelors =
student.getQualification("BACHELORS");
        if (!bachelors) return 0.0;

        double baseScore = normalizer->normalize(bachelors-
>getCgpaOrPercentage());

        const Qualification* gate = student.getQualification("GATE");
        if (gate && gate->isSpecialExamQualified()) {
            return (baseScore * 0.6) + (gate->getSpecialExamScore() * 0.4);
// 60% UG + 40% GATE weightage
        }
        return baseScore * 0.6;
    }
};

```

8. Program.h & Department.h

Ek `Department` me bohot saare `Program` (MCA, BCA, M.Tech) hote hain. `Program` class apne saath uska curriculum aur eligibility checker store karti hai.

```
// Program.h
#pragma once
#include <string>
#include <memory>
#include "Curriculum.h"
#include "EligibilityChecker.h"

using namespace std;

class Program {
    string name;
    string type;
    shared_ptr<Curriculum> curriculum;
    shared_ptr<EligibilityChecker> eligibilityChecker;

public:
    Program(string name, string type, shared_ptr<Curriculum> curriculum,
shared_ptr<EligibilityChecker> eligibilityChecker)
        : name(name), type(type), curriculum(curriculum),
eligibilityChecker(eligibilityChecker) {}

    string getName() const { return name; }
    string getType() const { return type; }
    shared_ptr<Curriculum> getCurriculum() const { return curriculum; }
    shared_ptr<EligibilityChecker> getEligibilityChecker() const { return
eligibilityChecker; }
};

// Department.h
#pragma once
#include <string>
#include <vector>
#include <memory>
#include "Program.h"

using namespace std;

class Department {
    string name;
    vector<shared_ptr<Program>> offeredPrograms;

public:
    Department(string name) : name(name) {}

    void addProgram(shared_ptr<Program> program) {
        offeredPrograms.push_back(program);
    }

    string getName() const { return name; }
};
```

9. AdmissionProcess.h

Yeh application process, merit ranking, aur seat limits ko check karke admissions grant karta hai.

```
#pragma once
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <algorithm>
#include "Program.h"
#include "Department.h"
#include "MeritCalculator.h"
#include "Student.h"

using namespace std;

class AdmissionProcess {
    int academicYear;
    shared_ptr<Program> program;
    shared_ptr<Department> department;
    int seatCapacity;
    shared_ptr<MeritCalculator> meritCalculator;
    vector<Student*> applicants;
    vector<Student*> admittedStudents;

public:
    AdmissionProcess(int academicYear, shared_ptr<Program> program,
shared_ptr<Department> department, int seatCapacity,
shared_ptr<MeritCalculator> meritCalculator)
        : academicYear(academicYear), program(program),
department(department), seatCapacity(seatCapacity),
meritCalculator(meritCalculator) {}

    // Student jab program ke liye apply kare
    void apply(Student& student) {
        auto checker = program->getEligibilityChecker();
        // Check karta hai agar eligible hai ya reject ho gaya
        if (checker && !checker->isEligible(student)) {
            cout << "Application Rejected for " << student.getName() << " -
> " << checker->getRejectionReason(student) << endl;
            return;
        }
        cout << "Application Accepted for " << student.getName() << " to "
<< program->getName() << endl;
        applicants.push_back(&student);
    }

    // Merit list banana aur admission dena seat availability ke hisab se
    void generateMeritListAndAdmit() {
        // Merit calculator ka use karke sorting (Descending order mein)
        sort(applicants.begin(), applicants.end(), [this](Student* s1,
```

```

Student* s2) {
    return meritCalculator->calculateMerit(*s1) > meritCalculator-
>calculateMerit(*s2);
    });

    cout << "--- Merit List for " << program->getName() << " (" <<
academicYear << ") ---" << endl;
    int count = 0;
    for (Student* s : applicants) {
        double score = meritCalculator->calculateMerit(*s);
        cout << s->getName() << " - Score: " << score << endl;
        if (count < seatCapacity) { // Agar seats hain tabhi admission
            admittedStudents.push_back(s);
            count++;
        }
    }
    cout << "-----" <<
endl;
}

// Admitted students ka list print karna
void printAdmittedStudents() const {
    cout << "Admitted Students to " << program->getName() << " (" <<
academicYear << "):" << endl;
    for (Student* s : admittedStudents) {
        cout << "- " << s->getName() << endl;
    }
}
};

```

10. main.cpp

Main file mein hum un saare classes ko use karte hain admission process ko test karne ke liye (BCA, MCA, MTech mein admissions).

```

#include <iostream>
#include <memory>
#include <vector>
#include <string>
#include "Qualification.h"
#include "Student.h"
#include "EligibilityChecker.h"
#include "ScoreNormalizer.h"
#include "IndianCGPANormalizer.h"
#include "PassThroughNormalizer.h"
#include "BasicEligibilityChecker.h"
#include "Curriculum.h"
#include "Program.h"
#include "Department.h"
#include "MeritCalculator.h"
#include "StandardMeritCalculator.h"

```

```

#include "GatePreferenceMeritCalculator.h"
#include "AdmissionProcess.h"

using namespace std;

int main() {
    cout << "=== Generalized Student Admission & Program Registration
System ===" << endl;

    auto scsitDept = make_shared<Department>("School of Computer Science &
IT (DAVV)");

    // Curriculum banaya gaya
    auto bcaCurriculum = make_shared<Curriculum>(6, 120, vector<string>
{"Programming in C", "Database Management", "Web Technologies"});
    auto mtechCsCurriculum = make_shared<Curriculum>(4, 80, vector<string>
{"Advanced ML", "Distributed Systems", "Cloud Computing"});
    auto mcaCurriculum = make_shared<Curriculum>(4, 80, vector<string>
{"Advanced Java", "Cloud Computing", "Software Engineering"});

    auto indianNormalizer = make_shared<IndianCGPANormalizer>();

    // UG aur PG ke liye eligibility check rules
    auto ugEligibility = make_shared<BasicEligibilityChecker>
("HIGH_SCHOOL", 50.0, indianNormalizer); // 12th me 50% chahiye
    auto pgEligibility = make_shared<BasicEligibilityChecker>("BACHELORS",
60.0, indianNormalizer); // Degree me 60% chahiye

    // Programs register karte hain department mein
    auto bca = make_shared<Program>("BCA", "UG", bcaCurriculum,
ugEligibility);
    auto mca = make_shared<Program>("MCA", "PG", mcaCurriculum,
pgEligibility);
    auto mtechCS = make_shared<Program>("M.Tech Computer Science", "PG",
mtechCsCurriculum, pgEligibility);
    auto intMCA = make_shared<Program>("Integrated MCA (BCA + MCA)", "UG",
make_shared<Curriculum>(10, 200, vector<string>{"Fundamentals of IT",
"AI"}), ugEligibility);

    scsitDept->addProgram(bca);
    scsitDept->addProgram(mca);
    scsitDept->addProgram(mtechCS);
    scsitDept->addProgram(intMCA);

    // Dummy Students banate hain different qualifications ke saath
    Student s1("S101", "Alice");
    s1.addQualification(Qualification("High School", 92.5, "DPS", 2023,
"HIGH_SCHOOL")); // Sirf 12th pass hai

    Student s2("S102", "Bob");
    s2.addQualification(Qualification("High School", 48.0, "KVS", 2023,
"HIGH_SCHOOL")); // Reject hoga due to < 50%

    Student s3("S103", "Charlie");

```

```
s3.addQualification(Qualification("High School", 85.0, "State Board",
2020, "HIGH_SCHOOL"));
s3.addQualification(Qualification("BCA", 8.5, "DAVV", 2023,
"BACHELORS")); // PG eligible

Student s4("S104", "David");
s4.addQualification(Qualification("High School", 90.0, "CBSE", 2019,
"HIGH_SCHOOL"));
s4.addQualification(Qualification("B.Tech", 9.2, "IIT", 2023,
"BACHELORS"));
s4.addQualification(Qualification("GATE", 75.0, "GATE Board", 2023,
"GATE", true, 75.0)); // GATE cleared

// Alag alag branches ke admission processes (seat limit aur merit
criteria apply kar rahe hain)
AdmissionProcess bcaAdmission(2024, bca, scsitDept, 1,
make_shared<StandardMeritCalculator>("HIGH_SCHOOL", indianNormalizer));
AdmissionProcess mcaAdmission(2024, mca, scsitDept, 1,
make_shared<StandardMeritCalculator>("BACHELORS", indianNormalizer));
AdmissionProcess mtechCsAdmission(2024, mtechCS, scsitDept, 2,
make_shared<GatePreferenceMeritCalculator>(indianNormalizer)); // Isme GATE
merit lagti hai

cout << "\n--- Applying to Programs ---" << endl;
// Students ko apply karwana program mein
bcaAdmission.apply(s1);
bcaAdmission.apply(s2); // Yeh print karega ki Bob reject ho gaya
mcaAdmission.apply(s3);
mtechCsAdmission.apply(s4);

cout << "\n--- Generating Merit Lists & Admissions ---" << endl;
// Selection process generate karna
bcaAdmission.generateMeritListAndAdmit();
bcaAdmission.printAdmittedStudents();
cout << endl;

mcaAdmission.generateMeritListAndAdmit();
mcaAdmission.printAdmittedStudents();
cout << endl;

mtechCsAdmission.generateMeritListAndAdmit();
mtechCsAdmission.printAdmittedStudents();
cout << endl;

return 0;
}
```